

Compte rendu n°3
Tp Tests de Logiciels
Tests automatisés avec JUnit et Spring

Nom:Rayen Bouyahia
Classe:L2CS01

1. Introduction

Ce TP porte sur la mise en place de tests automatisés dans un projet Java/Spring, en utilisant JUnit 5 comme framework de test. L'objectif est de comprendre comment tester correctement les différentes couches d'une application Spring — en particulier la couche Service et la couche DAO — tout en gérant convenablement le contexte Spring et les transactions.

L'architecture visée sépare strictement :

- La couche Service : contient la logique métier et gère les transactions.
- La couche DAO : responsable des accès à la base de données (JDBC ou Hibernate).

2. Test des beans de Service

2.1 Initialisation manuelle du contexte Spring

La première approche consiste à initialiser le contexte Spring directement dans le code de test, dans une méthode annotée `@BeforeAll`. Le bean à tester est ensuite récupéré depuis ce contexte.

```
@TestInstance(Lifecycle.PER_CLASS)
class ProductServiceTest {

    ProductService service;

    @BeforeAll
    void init() {
        ClassPathXmlApplicationContext context
            = new ClassPathXmlApplicationContext("application-config.xml");
        service = context.getBean(ProductService.class);
    }
}
```

Une méthode de test suit le pattern classique GIVE N / WHeN / THeN :

```
@Test
void getAll_should_work() {
    // GIVE N
    // WHeN
```

```
List<Product> products = service.getAll();
// THEN
assertThat(products).isEmpty();
}
```

2.2 Initialisation par annotation (Spring)

Plutôt que de gérer manuellement le contexte, on peut déléguer l'injection à Spring via `@ExtendWith` et `@ContextConfiguration`, puis injecter le bean avec `@Autowired` :

```
@ExtendWith(SpringExtension.class)
@ContextConfiguration(locations={"/application-config.xml"})
public class ProductServiceTest {

    @Autowired
    ProductService service;

    // méthodes de test...
}
```

2.3 Avec Spring Boot

Avec Spring Boot, l'annotation `@SpringBootTest` remplace `@ExtendWith` + `@ContextConfiguration`. Le reste du test est identique :

```
@SpringBootTest
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class ProductServiceBootTest {

    @Autowired
    ProductService service;

    // méthodes de test...
}
```

3. Test des beans DAO

Tester la couche DAO introduit une contrainte supplémentaire : chaque appel à un DAO doit s'exécuter dans le cadre d'une transaction. Il faut donc que la classe de test soit capable de démarrer et de terminer des transactions.

3.1 Gestion des transactions par programmation

En récupérant un `PlatformTransactionManager` depuis le contexte Spring, on peut encadrer manuellement chaque appel DAO dans une transaction :

```
@Test
void findByTitle_should_work() {
    TransactionStatus status = transactionManager.getTransaction(null);
    List<Product> products = productDao.findByTitleLike("TEST");
}
```

```

        transactionManager.commit(status);

        assertThat(products).isNotNull();
    }

```

Pour améliorer la lisibilité, on peut extraire ce mécanisme dans une méthode utilitaire utilisant une expression lambda (disponible depuis JDK 8) :

```

<T> T runInTransaction(Supplier<T> supplier) {
    TransactionStatus status = transactionManager.getTransaction(null);
    T result = supplier.get();
    transactionManager.commit(status);
    return result;
}

// Usage dans un test :
List<Product> products = runInTransaction(
    () -> productDao.findByTitleLike("TEST")
);

```

3.2 Gestion des transactions par annotation

La méthode la plus élégante, introduite avec Spring 2.5, consiste à annoter directement la classe de test avec `@Transactional`. Spring gère alors automatiquement le démarrage et le commit/rollback de la transaction pour chaque méthode de test :

```

@ExtendWith(SpringExtension.class)
@ContextConfiguration(locations={"/application-config.xml"})
@Transactional
public class ProductDaoTest {

    @Autowired
    ProductDao productDao;

    @Test
    void findByTitle_should_work() {
        List<Product> products = productDao.findByTitleLike("TEST");
        assertThat(products).isNotNull();
    }
}

```

3.3 Comportement des méthodes de cycle de vie

Lorsque `@Transactional` est utilisé sur la classe de test, il est important de comprendre dans quel contexte transactionnel s'exécutent les méthodes de cycle de vie :

Annotation	Dans la transaction ?	Remarque
<code>@BeforeEach</code> / <code>@AfterEach</code>	Oui	Même transaction que <code>@Test</code>
<code>@BeforeAll</code> / <code>@AfterAll</code>	Non	S'exécutent hors transaction
<code>@BeforeTransaction</code>	Non	Juste avant le démarrage
<code>@AfterTransaction</code>	Non	Juste après le commit/rollback

4. Bilan & Points clés

Ce TP a permis de maîtriser les différentes stratégies de test dans un contexte Spring :

- Initialisation manuelle avec `ClassPathXmlApplicationContext` pour un contrôle total.
- Injection automatique avec `@ExtendWith` + `@ContextConfiguration` pour plus de clarté.
- Simplification avec `@SpringBootTest` dans un projet Spring Boot.
- Gestion des transactions par programmation (`runInTransaction`) ou par annotation (`@Transactional`).